



Esta guía de estudio está diseñada para orientarte en tus primeros pasos con Python.

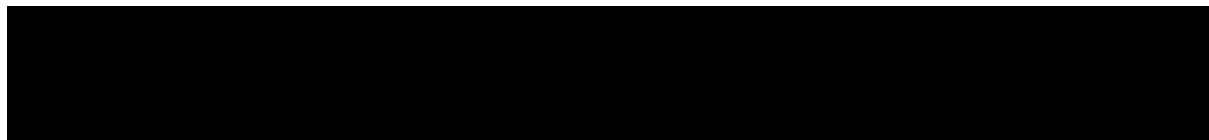
El Corazón de Python: Intérprete Interactivo vs. Scripts (.py)

1. Introducción: Las Dos Caras de Python

Python es un lenguaje diseñado para la claridad y la eficiencia, nuestro objetivo es que entiendas que no solo estás aprendiendo a escribir líneas de código, sino a elegir la herramienta adecuada para cada tarea, existen dos formas fundamentales de interactuar con el motor de Python: una es una conversación fluida y en tiempo real, y la otra es una construcción meticulosa y duradera.

Idea Maestra: El modo interactivo es tu laboratorio para resultados inmediatos, mientras que los archivos de script (.py) son los cimientos permanentes donde vive tu código real.

Dominar ambas modalidades es lo que separa a un estudiante de un practicante informado, la interactividad te permite fallar rápido y aprender en segundos, mientras que los scripts te permiten escalar tus soluciones.



2. El Intérprete Interactivo: Tu Laboratorio de Experimentos

El modo interactivo, también conocido como **REPL** (Read-Eval-Print Loop), es un entorno donde el código se ejecuta línea por línea, es la herramienta perfecta para probar una nueva función, verificar la sintaxis de una operación matemática o explorar bibliotecas sin la fricción de crear un archivo.

Cómo iniciar y salir del intérprete

Dependiendo de tu sistema y de cómo instalaste Python, los comandos varían. Un profesional siempre verifica su versión primero:

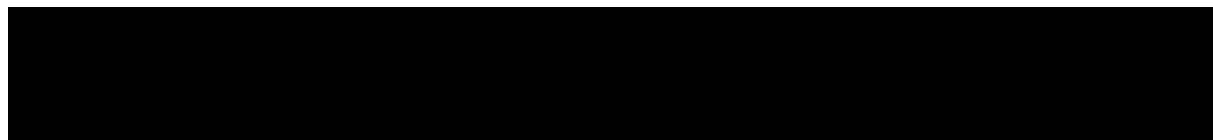
- **En Mac, Linux o WSL:** El estándar moderno es usar el comando **python3**.
- **En Windows:**
 - Si instalaste desde la Microsoft Store, usa **python**.
 - Si instalaste desde python.org, puedes usar el lanzador **py** (o **py -3.13** para una versión específica). Ten en cuenta que el comando **py** no funciona con la versión de la Store.

Acción	Comando (Unix / Mac / WSL)	Comando (Windows)
Entrar al modo interactivo	python3	python o py
Salir del intérprete	Ctrl-D	Ctrl-Z (seguido de Enter)

Beneficios para tu aprendizaje:

1. **Feedback Instantáneo:** Obtienes respuestas de Python en milisegundos.
2. **Validación de Ideas:** ¿No estás seguro de cómo funciona un método de cadenas? Pruébalo en el REPL antes de llevarlo a tu código principal.
3. **Exploración Dinámica:** Ideal para usar durante las clases mientras sigues una demostración en vivo.

Nota del Mentor: Aunque el intérprete es ágil, es volátil. Todo lo que escribas aquí se perderá al cerrar la ventana.



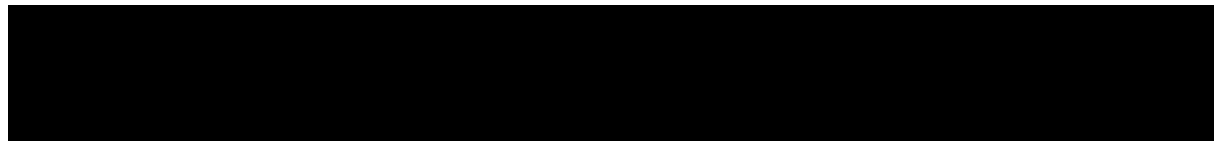
3. Archivos de Script (.py): Construcciones Permanentes

Para crear aplicaciones reales, necesitamos **persistencia**, un script de Python no es más que un archivo de **texto plano** con la extensión **.py**. Esta extensión

le indica al sistema operativo y al intérprete que el contenido debe ser tratado como instrucciones de programación.

¡ADVERTENCIA TÉCNICA! Nunca utilices procesadores de texto como Microsoft Word, Wordpad o el Bloc de Notas de Windows, estos programas insertan "caracteres invisibles" y formatos ocultos que Python no puede procesar, necesitamos texto puro, utiliza editores profesionales:

- **Windows:** Notepad++ o JEdit.
- **Mac:** BBEdit o JEdit.
- **Cualquier SO:** Visual Studio Code (altamente recomendado).



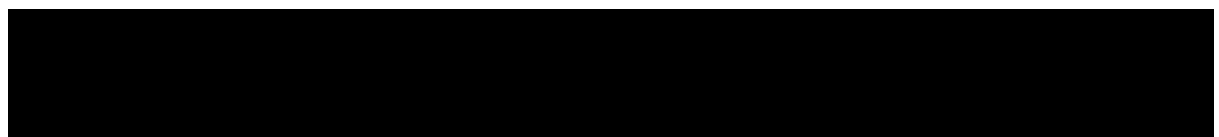
4. El Ciclo Maestro: Editar y Ejecutar

El flujo de trabajo profesional se basa en un ciclo constante de cambios y verificaciones, para ser eficiente, mantén dos ventanas abiertas simultáneamente: tu editor de texto y tu terminal.

El flujo paso a paso:

1. **Editar:** Abre tu archivo (ej. hello.py) y realiza un cambio, por ejemplo, cambia `print('Hola')` por `print('Howdy')`.
2. **Guardar:** Este es el paso que más olvidan los principiantes, si no guardas, la terminal ejecutará la versión anterior.
3. **Ejecutar:** En la terminal, escribe el comando para correr el archivo.
4. **Pro-Tip de Eficiencia (La Flecha Arriba):** No vuelvas a escribir el comando, en la terminal, presiona la **tecla de flecha hacia arriba** para recuperar el último comando ejecutado y presiona **Enter**, esto ahorra horas de trabajo acumuladas.

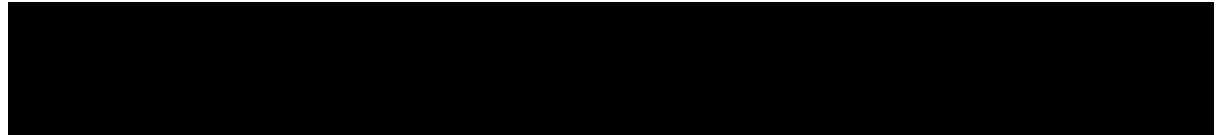
Comparación del Ciclo de Trabajo | Acción en el Editor | Comando en la Terminal | Resultado Esperado |
| :--- | :--- | :--- |
| Cambiar texto a "Howdy" | **python3 hello.py** | Se imprime "Howdy" |
| Agregar `print('yay!')` | **python3 hello.py** | Se imprimen ambas líneas |
| Corregir un error de sintaxis | (Flecha Arriba + Enter) | El programa corre sin errores.



5. Configuración Óptima para el Aprendizaje

Para evitar errores que parecen "misteriosos", configura tu editor siguiendo estas reglas de oro de la industria:

- **Sangría (Indentation):** Python usa espacios para estructurar el código, configura tu editor para que la tecla **Tab** inserte **espacios**, los tutoriales de Google recomiendan **2 espacios** para mantener la consistencia, aunque 4 es otra opción común.
- **Sangría Automática:** Asegúrate de que al presionar **Enter**, el cursor se alinee automáticamente con la línea anterior.
- **Finales de Línea Unix (LF):** Los archivos deben guardarse con el formato **Unix (LF - Line Feed)**. Si al ejecutar un script ves el error **Unknown option: -**, es casi seguro que tu editor guardó el archivo con finales de línea de Windows.
- **Rutas en Windows (Raw Strings):** Windows usa la barra diagonal inversa (\) para las rutas, pero Python la interpreta como una "secuencia de escape" (ej. \n significa nueva línea). Para evitar errores, antepón una **r** antes de las comillas: **r"C:\Users\Docs"**. Esto le dice a Python que es una cadena "cruda" (raw) y que no procese caracteres especiales.



6. Cuadro Comparativo: ¿Qué herramienta usar y cuándo?

Característica	Intérprete Interactivo (REPL)	Scripts (.py)
Cuándo usarlo	Experimentos rápidos y pruebas de sintaxis.	Desarrollo de ejercicios, tareas y aplicaciones.
Persistencia	Volátil (Se borra al cerrar).	Permanente (Se guarda en disco).

Ventajas	Resultados inmediatos; sin archivos intermedios.	Código editable, compartible y reutilizable.
Limitaciones	Difícil de editar bloques largos de código.	Requiere el ciclo "Guardar y Ejecutar".
Ejemplo de Comando	<code>>>> print("Hi")</code>	<code>python3 hello.py</code>

Recuerda: la maestría técnica nace de la repetición. **Realiza experimentos rápidos en el intérprete para entender el "cómo", pero construye tus programas reales en archivos .py para dominar el "qué".** ¡A programar!

